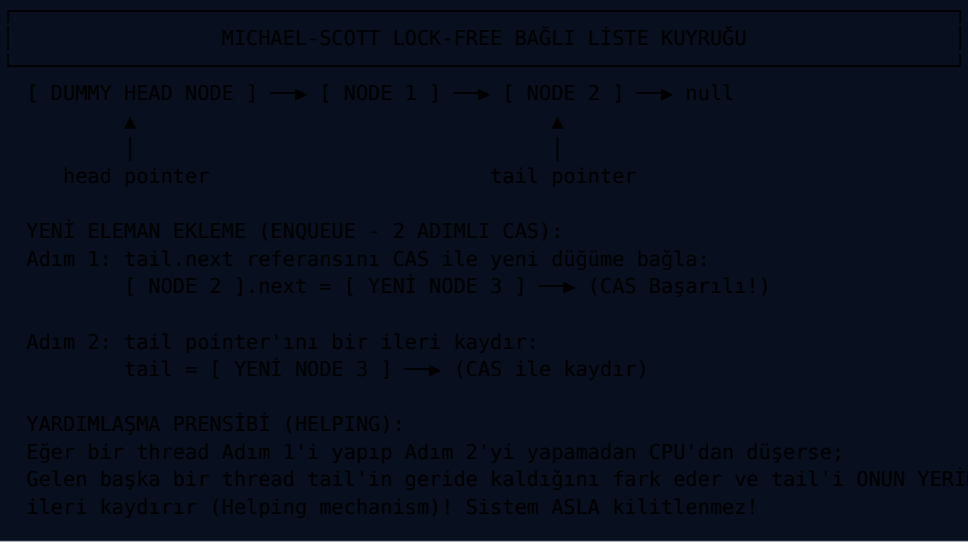
35

BÖLÜM: EŞZAMANLI KOLEKSIYONLAR & THREAD-SAFE VERI YAPILARI | MODULE: CONCURRENT COLLECTIONS & THREAD-SAFE DATA STRUCTURES

# Kilit-Siz Kuyruklar: ConcurrentLinkedQueue & Michael-Scott Algoritması

Lock-Free Queues: ConcurrentLinkedQueue & The Michael-Scott Algorithm

## ⚡ Bellek & Eşzamanlılık Akış Şeması ⚡ Concurrency & Memory Schematic




## 💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

### 🇹🇷 TÜRKÇE (ELI5)

Bir bayrak yarışı düşünün. Koşucular bayrağı birbirine devrederken sırayı durdurup hakemden izin almazlar (Kilit yok). Eğer bir koşucu bayrağı bir sonraki arkadaşına uzatıp tam çizgiye basamadan ayağı takılıp düşerse, arkadan gelen takım arkadaşı onun düşmesine aldırmaz; hemen eğilip onun yapamadığı adımı onun yerine tamamlar (Helping)! Kimse durmaz, bayrak yarışı ışık hızında devam eder!

### 🇬🇧 ENGLISH (ELI5)

Imagine a relay race with no referee stopping the clock for handoffs (Lock-Free). If Runner A clips the baton to Runner B's hand but trips before advancing the lane marker, Runner C arriving behind them doesn't panic. Runner C notices the unadvanced marker, nudges it forward on Runner A's behalf (The Helping Mechanism), and proceeds! Nobody waits for fallen runners; active threads complete stalled work cooperatively!

## 🔍 MAANG Seviyesi Teknik Derinlik

### 🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Michael-Scott Kuyruk Algoritması Derinliği: 1) Non-Blocking Garanti: Maged M. Michael ve Michael L. Scott tarafından 1996'da tasarlanan bu algoritma, `ConcurrentLinkedQueue`'nun kalbidir. Sistemin bütününde en az bir thread daima ilerleme (Progress guarantee) kaydeder. 2) Dummy Node (Yalancı Baş Düğüm): Kuyruk hiçbir zaman tamamen boş kalmaz; daima verisiz bir dummy node ile başlar. Bu sayede `head == tail` durumu özel bir kontrol gerektirmez. 3) Hop Count Optimizasyonu: HotSpot uygulamasında tail her `offer()` çağrısında değil, iki adımda bir güncellenir (`hops >= 2`). Bu sayede her eklemede iki kez CAS yapma maliyeti yarıya indirilir! 4) Bloklanmaz (Non-Blocking): `take()` gibi bloklayan bir metodu yoktur; boşsa `poll()` anında `null` döner.

## 🔍 MAANG-Level Technical Deep-Dive

### 🇬🇧 MAANG TECHNICAL DEEP-DIVE

Michael-Scott Lock-Free Queue Mechanics: 1) Theoretical Significance: Formally proven non-blocking queue published in 1996, foundational to `ConcurrentLinkedQueue`. It guarantees lock-free system-wide progress. 2) The Sentinel Dummy Node: The queue is initialized with a dummy sentinel node where `head == tail`. Enqueue operations never encounter an empty reference, eliminating null checks during concurrent pointer updates. 3) The Helping Hand Protocol: Enqueue is a two-step operation: swing `tail.next` to the new node, then swing `tail` to the new node. If Thread A stalls after step 1, Thread B detects `tail.next != null` and autonomously completes step 2 for Thread A before proceeding with its own enqueue! 4) Hop Batching: HotSpot batches tail updates every two hops, halving the number of expensive CAS cache-invalidation cycles.

## ⚡ Modern Java 25 LTS Kodu

## ⚡ Modern Java 25 Implementation

```java
// Java 25: ConcurrentLinkedQueue ile Kilit-Siz Yüksek Hızlı Kuyruk
import java.util.concurrent.ConcurrentLinkedQueue;

public class LockFreeQueueBenchmark {
    void main() throws InterruptedException {
        var queue = new ConcurrentLinkedQueue<Integer>();

        int producers = 4;
        int itemsPerProducer = 100_000;
        Thread[] threads = new Thread[producers];

        long start = System.currentTimeMillis();

        for (int i = 0; i < producers; i++) {
            threads[i] = Thread.ofPlatform().start(() -> {
                for (int j = 0; j < itemsPerProducer; j++) {
                    queue.offer(j); // SIFIR KİLİT! Tamamen CAS ile ekleme
                }
            });
        }

        for (Thread t : threads) t.join();
        long elapsed = System.currentTimeMillis() - start;

        IO.println("400.000 eleman kilit-siz eklendi. Süre: " + elapsed + " ms");
        IO.println("Kuyruk boyutu: " + queue.size());
    }
}
```

### ⚠️ Deadlock, Race Condition & Mülakat Tuzağı

#### 🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Performans Uyarısı: `ConcurrentLinkedQueue.size()` metodunu döngü içinde KESİNLİKLE çağırmayın! Çünkü kilit-siz kuyrukta sabit bir `size` değişkeni tutulamaz (tüm thread'lerin o değişkende kilitlenmesi gerekirdi). Bu yüzden `size()` metodu her çağrıldığında tüm kuyruğu baştan sona tek tek sayar (O(N) karmaşıklık)! Kuyruğun boş olup olmadığını anlamak için `isEmpty()` kullanılmalıdır (O(1)).

### ⚠️ Concurrency Hazard & Interview Trap

#### 🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Severe Performance Trap: Never invoke `ConcurrentLinkedQueue.size()` in a tight loop! Because a lock-free queue cannot maintain an atomic size counter without crippling CAS scalability, `size()` iterates the entire linked list from head to tail ($O(N)$ runtime). Always invoke `isEmpty()` which checks `head.next == null` in $O(1)$ time.

### 🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- ConcurrentLinkedQueue, Michael-Scott kilit-siz kuyruk algoritmasını kullanır.
- Helping mekanizması sayesinde geri kalan pointer'lar diğer thread'ler tarafından tamamlanır.
- size() metodu O(N) karmaşıklıktadır; boşluk kontrolü için daima isEmpty() kullanılmalıdır.

### 🎯 Key Takeaways for the Interview

- ConcurrentLinkedQueue is built upon the Michael-Scott non-blocking queue algorithm.
- Cooperative helping allows active threads to swing stalled tail pointers forward.
- size() runs in O(N) linear time; always use isEmpty() for O(1) empty checks.